

問：

SIC/XE 指令的語法大致上如下：

[label] [+] mnemonic [[prefix] operand1 [, operand2]] [. comment]

中括號表示可選項目，我是否有描述錯誤？

該如何用 C 解析？

你的描述**非常精準**，完美捕捉了 SIC/XE 組合語言指令的基本結構！中括號 [] 代表可選項目，這在描述語法時是非常標準且正確的寫法。

為了讓解析更嚴謹，我幫你補充幾個 SIC/XE 語法在實際文字檔中的**細節特性**，這會直接影響我們後續用 C 語言實作解析器 (Parser) 的邏輯：

1. **標籤 (Label) 的判定**：在標準的 SIC/XE 原始碼中，標籤必須從**第一行 (Column 1 ，也就是索引 0) 開始寫**。如果該行的第一個字元是空白 (Space 或 Tab)，就代表該行「沒有標籤」，直接從助憶碼 (Mnemonic) 開始。
2. **整行註解**：如果該行的第一個字元是 `.`，則代表整行都是註解。
3. **加號與前綴**：`+` (Format 4) 通常會緊貼著 Mnemonic (例如 `+LDT`)；而 `@`、`#`、`=` 等前綴通常會緊貼著 Operand1 (例如 `#4096`)。
4. **逗號 (Comma)**：通常緊貼在 Operand1 和 Operand2 之間 (例如 `BUFFER,X`)，不一定有空格。

如何用 C 語言解析？

在 C 語言中，不建議單純使用 `strtok` 來切空白，因為 `strtok` 會自動忽略連續的空白字元，這會導致我們無法判斷「第一個欄位到底是標籤還是助憶碼」。

比較穩健的作法是**手動搭配指標與 `sscanf`**，根據字串的位置來解析。以下提供一個清晰且健壯的 C 語言範例程式碼：

C

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>
#include <stdbool.h>
```

```
// 定義一個結構來儲存解析後的指令
```

```

typedef struct {
    char label[32];
    bool is_format4;
    char mnemonic[32];
    char operand1[32];
    char operand2[32];
    char comment[128];
} Instruction;

void parse_sicxe_line(char *line, Instruction *inst) {
    // 1. 初始化結構與去除行尾換行符號
    memset(inst, 0, sizeof(Instruction));
    line[strcspn(line, "\r\n")] = '\0';

    if (strlen(line) == 0) return; // 空行

    // 2. 處理整行註解 (第一個字元是 '.')
    if (line[0] == '.') {
        strcpy(inst->comment, line);
        return;
    }

    char *ptr = line;

    // 3. 解析 Label (如果第一個字元不是空白，就是 Label)
    if (!isspace((unsigned char)*ptr)) {
        sscanf(ptr, "%31s", inst->label);
        while (*ptr && !isspace((unsigned char)*ptr)) ptr++; // 指標推移過 Label
    }

    // 略過空白
    while (*ptr && isspace((unsigned char)*ptr)) ptr++;

```

```

// 4. 解析 Mnemonic (包含 + 號判斷)
if (*ptr) {
    char temp_mnem[32] = {0};
    sscanf(ptr, "%31s", temp_mnem);

    if (temp_mnem[0] == '+') {
        inst->is_format4 = true;
        strcpy(inst->mnemonic, temp_mnem + 1); // 存入時去掉 '+'
    } else {
        strcpy(inst->mnemonic, temp_mnem);
    }
    while (*ptr && !isspace((unsigned char)*ptr)) ptr++; // 指標推移過 Mnemonic
}

// 略過空白
while (*ptr && isspace((unsigned char)*ptr)) ptr++;

// 5. 解析 Operands
if (*ptr && *ptr != '.') {
    char temp_operand[64] = {0};
    sscanf(ptr, "%63s", temp_operand);
    while (*ptr && !isspace((unsigned char)*ptr)) ptr++; // 指標推移過 Operands

    // 處理 operand1, operand2 (用逗號分割)
    char *comma = strchr(temp_operand, ',');
    if (comma) {
        *comma = '\0'; // 切斷字串
        strcpy(inst->operand1, temp_operand);
        strcpy(inst->operand2, comma + 1);
    } else {
        strcpy(inst->operand1, temp_operand);
    }
}
}

```

```

// 略過空白
while (*ptr && isspace((unsigned char)*ptr)) ptr++;

// 6. 解析行尾註解
if (*ptr) {
    strcpy(inst->comment, ptr);
}
}

// 測試用主程式
int main() {
    char test_lines[][128] = {
        "COPY  START 1000      . COPY FILE PROGRAM",
        "    +LDT  #4096",
        "CLOOP  LDA  BUFFER,X    . 讀取字元",
        ". THIS IS A FULL LINE COMMENT",
        "    RSUB"
    };

    int num_lines = sizeof(test_lines) / sizeof(test_lines[0]);
    Instruction inst;

    for (int i = 0; i < num_lines; i++) {
        printf("原始行: %s\n", test_lines[i]);
        parse_sicxe_line(test_lines[i], &inst);

        printf("-> Label: '%s'\n", inst.label);
        printf("-> Format 4: %s\n", inst.is_format4 ? "Yes" : "No");
        printf("-> Mnemonic: '%s'\n", inst.mnemonic);
        printf("-> Operand1: '%s'\n", inst.operand1);
        printf("-> Operand2: '%s'\n", inst.operand2);
        printf("-> Comment: '%s'\n", inst.comment);
    }
}

```

```
    printf("-----\n");  
}  
  
return 0;  
}
```

程式碼設計重點解說：

- **指標掃描 (Pointer Scanning)**：利用 `char *ptr` 像游標一樣在字串上前進。每讀取完一個區塊 (例如 Label)，就將指標推移到下一個空白處，再略過空白。這種方式比嚴格切割字串更具有容錯性。
- **`isspace((unsigned char)*ptr)`**：這是在 C 語言判斷空白字元 (包含 Space 和 Tab) 的最佳做法，特別強制轉型為 `unsigned char` 以避免某些系統字元集造成的 Undefined Behavior。
- **Format 4 分離**：程式會偵測 Mnemonic 是否以 + 開頭，如果是，就會將布林值 `is_format4` 設為 true，並把純 Mnemonic (去掉加號) 存起來，方便你後續去查 Opcode 表。
- **逗號分割 (Comma splitting)**：讀取 Operand 區塊後，利用 `strchr` 尋找有沒有,。如果有，就把, 換成字串結尾符號 `\0`，這樣就能輕鬆分離出 `operand1` (例如 BUFFER) 與 `operand2` (例如 X)。